

We introduced the concept of trigger variables:

- NEW
- OLD
- TG_OP

As well as data manipulation-based triggers, we briefly introduced the PostgreSQL event triggers that allow developers and database administrators to have more control over firing and executing functions.

We have come to understand that triggers are extremely complex event handlers. In this chapter, we started to show the power of the tools made available to the PostgreSQL DBA; in the next chapter, we will talk about partitioning, and we will utilize the topics covered in this chapter to do so.

Verify your knowledge

- What is the NEW record?

The NEW record is the record that is going to be processed before an INSERT statement or an UPDATE statement, for example:

```
insert into mytable(id,city_name) values (1,'New York')
```

```
NEW.id = 1  
NEW.city_name = 'New York'
```

See the section *Exploring rules in PostgreSQL* for more details.

- Can we execute an INSERT on two tables in a single transaction using rules?

Yes, we can; we can make it using the ALSO clause. See the section *Exploring rules in PostgreSQL* for more details.

- Can we make all the things we do with rules using triggers?

Yes, we can; by using triggers, we can make all the things we do with rules and more. See the section *Managing triggers in PostgreSQL* for more details.

- Can we know if a trigger has been fired from an INSERT event, from an update EVENT, or from a DELETE event?

Yes, we can, using the TG_OP variable. See the section *Managing triggers in PostgreSQL* for more details.

- Can we write an audit procedure that informs us when a DDL has been executed?

Yes, we can, using event triggers. See the section *Event triggers* for more details.

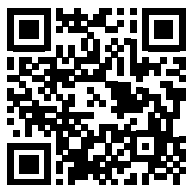
References

- PostgreSQL rules on the INSERT, UPDATE, and DELETE official documentation: <https://www.PostgreSQL.org/docs/current/rules-update.html>
- PostgreSQL trigger functions official documentation: <https://www.PostgreSQL.org/docs/current/plpgsql-trigger.html>
- PostgreSQL ALTER TRIGGER official documentation: <https://www.PostgreSQL.org/docs/current/sql-altertrigger.html>
- PostgreSQL DROP TRIGGER official documentation: <https://www.PostgreSQL.org/docs/current/sql-droptrigger.html>
- PostgreSQL EVENT TRIGGER official documentation: <https://www.postgresql.org/docs/current/functions-event-triggers.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



9

Partitioning

In the previous chapter, we talked about rules and triggers. In this chapter, we will talk about partitioning. Partitioning is a technique that allows us to split a huge table into smaller tables to make queries more efficient. In this chapter, we will see how we can partition data, and, in some cases, how we can use the rules and triggers seen in the previous chapter to make partitioning possible. We will start by introducing the basic concepts of partitioning, and then we will see the possibilities PostgreSQL offers to implement partitioning.

This chapter will cover the following topics:

- Basic concepts
- Partitioning using table inheritance
- Declarative partitioning

Technical requirements

The chapter examples can be run on the `chapter_09` Docker image that you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>. For installation and usage instructions for the Docker images for this book, please refer to *Chapter 1, Introduction to PostgreSQL*.

Basic concepts

First of all, let's try to understand why we have to partition data. We should start by saying that a common constant of all databases is that their size always grows. It is, therefore, possible that a database, after a few months of growth, can reach a size of gigabytes, terabytes, or even petabytes.

Another thing we must always keep in mind is that not all tables grow at the same rate or to the same level; some tables are bigger than others and some indexes too are bigger than other indexes.

We also need to know that there is a part of our server's RAM, shared among all the PostgreSQL processes, that is used to manage the data present in tables. This part of the server's RAM is called `shared_buffers`.

The way PostgreSQL works is as follows:

1. Data is taken from hard disks.
2. Data is placed in shared buffers.
3. Data is processed in shared buffers.
4. Data is downloaded to disks.

Typically, in a dedicated server only for PostgreSQL, the size of `shared_buffers` is about one-third or one-quarter of the total server RAM. A useful link to set some PostgreSQL configuration parameters (including a recommended size for `shared_buffers`) is <https://pgtune.leopard.in.ua>.

When a table grows excessively compared to the `shared_buffers` size, there is a possibility that performance will decrease. In this case, partitioning data can help us. Partitioning data means splitting a very large table into smaller tables in a way that is transparent to the client program. The client program will think that the server still has only a single table, but having smaller tables also means having smaller indexes that have higher chances of staying in memory, which in turn increases data performance; moreover, having smaller tables means that the vacuum processes works on smaller tables, which minimizes the execution time of the vacuum processes. Finally, when running a vacuum full, the disk space used by the table is doubled, therefore having many small tables instead of one large one significantly reduces any impact from this issue. Data partitioning can be done in two ways:

- Using table inheritance (the only possible way for PostgreSQL < 10)
- Using declarative partitioning (the best way starting from version 10)

After figuring out when it is recommended to partition data, let's see what types of table partitioning are possible. PostgreSQL supports three types of declarative partitioning:

- Range partitioning
- List partitioning
- Hash partitioning

We will now describe these three methods in detail.

Before starting, remember to start the Docker container named `chapter9`, as shown below:

```
$ bash run-pg-docker.sh chapter_09
postgres@learn_postgresql:~$ psql -U forum forumdb
```

Range partitioning

Range partitioning is where the table is divided into “intervals.” The intervals must not overlap and the range is defined through the use of a field or a set of fields. For further information, see <https://www.postgresql.org/docs/current/ddl-partitioning.html>.

Let’s look at an example of the definition of range partitioning. Suppose we have this table:

field date	field_value
2023-03-01	1
2023-03-02	10
2023-04-01	12
2023-04-15	1

Table 9.1: The table before range partitioning

Now consider that we want to split this table into two tables. The first table (**TABLE A**) will contain all the records with a `field_date` value between 2023-03-01 and 2023-03-31, and the second table (**TABLE B**) will contain all the records with a `field_date` value between 2023-04-01 and 2023-04-30. So, our goal is to have two tables as follows:

field date	field_value
2023-03-01	1
2023-03-02	10

Table 9.2: Table A

field date	field_value
2023-04-01	12
2023-04-15	1

Table 9.3: Table B

What we have seen is an example of partitioning by range. This is useful when we have large tables in which the data can be divided by time range, for example, turnover, audit tables, or log tables.

List partitioning

In list partitioning, the table will be partitioned using a list of values.

Let's look at an example of the definition of list partitioning. Suppose we have this table:

field_state	field_city
United States	Washington
United States	San Francisco
Italy	Rome
Japan	Tokyo

Table 9.4: The table before list partitioning

Suppose now that we want to split this table into n tables, with one table for each state. The first table (**TABLE A**) will contain all the records with a `field_state` value equal to United States, the second table (**TABLE B**) will contain all records with a `field_state` value equal to Italy, and the third table (**TABLE C**) will contain records with a `field_state` value equal to Japan. So, our goal is to have three tables as follows:

field_state	field_city
United States	Washington
United States	San Francisco

Table 9.5: Table A

field_state	field_city
Italy	Rome

Table 9.6: Table B

field_state	field_city
Japan	Tokyo

Table 9.7: Table C

This is an example of partitioning by list. This is useful when we have large tables where the data can be divided by a single field, such as a city or state field in a telephone directory, or in a customer list.

Hash partitioning

Using hash partitioning, the table will be partitioned using hash values to split data into different tables.

Let's look at an example of hash partitioning. Suppose we have this table:

field date	field_value
2023-03-01	1
2023-03-02	1
2023-04-01	2
2023-04-15	2

Table 9.8: The table before hash partitioning

Suppose now that we have a hash function that transforms a date into a hash value; for example, let's consider `del mod operator (%)`:

- `hash(1) = 1`
- `hash(1) = 1`
- `hash(2) = 0`
- `hash(2) = 0`

So, after the partitioning process, we will have two tables:

field date	field_value
2023-03-01	1
2023-03-02	1

Table 9.9: Table A

field date	field_value
2023-04-01	2
2023-04-15	2

Table 9.10: Table B

This is an example of partitioning by hash.

In the following sections we will see how PostgreSQL implements list, range, and hash partitioning, but before that, let's spend some time talking about table inheritance.

For further information about partitioning see <https://www.postgresql.org/docs/current/ddl-partitioning.html>.

Table inheritance

Another topic that we must look at is the inheritance of tables. PostgreSQL employs the concept of inheritance from databases to objects. The concept is very simple and can be summarized as follows: suppose we have two tables, **TABLE A** and **TABLE B**. If we define **TABLE A** as a parent table and **TABLE B** as the child table, this means that all the records in **TABLE B** will be accessible from **TABLE A**.

Let's now try to give an example of what we have just described:

1. Let's define two tables.

The first table, the parent table, is defined as follows:

```
forumdb=> create table table_a (
    pk integer not null primary key,
    tag text,
    parent integer);
CREATE TABLE
```

And the second table, the child table, is defined as follows:

```
forumdb=> create table table_b () inherits (table_a);
CREATE TABLE

forumdb=> alter table table_b add constraint table_b_pk primary
key(pk);
ALTER TABLE
```

2. The child table inherits all the fields from the parent table. The parent table is as seen here:

```
forumdb=> \d table_a;
               Table "forum.table_a"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 pk      | integer |           | not null |
```

```

tag      | text      |          |          |

parent   | integer    |          |          |

```

Indexes:

"table_a_pkey" PRIMARY KEY, btree (pk)

Number of child tables: 1 (Use \d+ to list them.)

And for more details, let's use the \d+ command:

```

forumdb=> \d+ table_a;
           Table "forum.table_a"
  Column | Type      | Collation | Nullable | Default | Storage |
Compression | Stats target | Description
-----+-----+-----+-----+-----+-----+-----
pk       | integer   |           | not null |         | plain   |
|         |           |           |         |         |         |

tag      | text      |           |         |         | extended |
|         |           |           |         |         |         |

parent   | integer   |           |         |         | plain   |
|         |           |           |         |         |         |

Indexes:
    "table_a_pkey" PRIMARY KEY, btree (pk)
Child tables: table_b
Access method: heap

```

In this last table, we can see that table_b is a child table of table_a.

3. Let's do the same for the table called table_b:

```

forumdb=> \d table_b;
           Table "forum.table_b"

```